Sieci neuronowe – wykłady 3 i 4

Nauka od zera: wiele klas (one-hot, softmax), propagacja wsteczna rozpisana na pojedyncze wagi, wersja z MSE, wersja macierzowa, funkcje aktywacji, zanikający i eksplodujący gradient, walidacja, bias i wariancja, przeuczenie.

Jak czytać: każdy wzór z wykładu jest rozpisany linijka po linijce. Pod wzorem jest ramka „co oznacza każdy symbol”, potem „po ludzku” i „na liczbach”. W przykładach wszystkie dane są wypisane na miejscu. Kolory ramek są takie same jak w NAUKA_cw3: szara – cytat ze slajdu, niebieska – wzór, przerywana – symbole, żółta – po ludzku, zielona – liczby, fioletowa – sprawdź się, czerwona – uwaga.

Plan nauki

rozdział	co	po czym poznasz, że umiesz	slajdy
0. Notacja wykładu	co znaczą p, k, j, i, górne indeksy, μ, t	czytasz $w_{kj}^{2}(t+1)$ na głos	W3: 6
1. Przejście w przód	przykład ze slajdów przeliczony ręcznie	dostajesz 0,62 i 0,83 jak na slajdzie	W3: 30–35
2. Wiele klas	one-hot, softmax, entropia krzyżowa	liczysz softmax z $[3, 1, -3]$	W3: 3, 5, 10
3. Warstwa wyjściowa	pochodna softmaxa, skąd $\hat{y} - y$	wyprowadzasz to w 5 linijkach	W3: 7–15
4. Warstwa ukryta	skąd $\delta_j^1 = f'(z_j) \sum_k \delta_k w_{kj}$	tłumaczysz, czemu jest suma po k	W3: 16–19
5. Pełny przykład	forward, oba δ, nowe wagi, sprawdzenie	robisz go sam na kartce	–
6. Wersja z MSE	co się zmienia, gdy koszt to MSE	mówisz, czemu MSE + sigmoida uczy się wolno	W4: 4–14
7. Zapis macierzowy	ten sam algorytm na macierzach	piszesz 4 linijki algorytmu z pamięci	W4: 17–19
8. Algorytm, SGD, epoki	dwie fazy, po wzorcu / po epoce, kiedy stop	wymieniasz 3 warunki zatrzymania	W3: 21–25, W4: 15, 21
9. Funkcje aktywacji	sigmoida, tanh, ReLU, Leaky ReLU, softplus	podajesz każdą i jej pochodną	W3: 20, W4: 23–31
10. Zły gradient	zanikający, eksplodujący, obcinanie	liczysz $0{,}25^{10}$ i wiesz, co to znaczy	W4: 25–29
11. Walidacja	zbiór uczący / walidujący / testowy, walidacja krzyżowa	mówisz, czemu nie stroić na teście	W3: 26–29, W4: 52–54
12. Bias i wariancja	rozkład błędu, przeuczenie, niedouczenie	liczysz bias² i wariancję z 3 modeli	W4: 32–57
13–15. Powtórka	ściąga, pytania, rozwiązania	odpowiadasz bez notatek	–

0. Notacja wykładu: co znaczy każda litera

Wykład pisze wszystko **dla pojedynczych wag** i używa dużo indeksów. Najpierw rysunek sieci, której dotyczą wszystkie wzory (slajd W3: 6):

```
 wejście x_p (N liczb)  →  warstwa ukryta (L neuronów)  →  warstwa wyjściowa (M neuronów)
   x_p1 … x_pi … x_pN        a¹_p1 … a¹_pj … a¹_pL            ŷ_p1 … ŷ_pk … ŷ_pM
           waga w¹_ji łączy wejście i z neuronem ukrytym j
                                  waga w²_kj łączy neuron ukryty j z neuronem
wyjściowym k
```

symbol	czytasz	co to jest
p	„numer wzorca”	który przykład ze zbioru uczącego liczymy (wzorzec = jeden przykład, np. jeden obrazek). P – liczba wszystkich wzorców.
$\mathbf{x}_p = (x_{p1}, \ldots, x_{pN})^T$	„wektor wejściowy wzorca p ”	N liczb wejściowych. ^{T} znaczy: zapisany pionowo (kolumna).
i	indeks wejścia	$i = 1, \ldots, N$
j	indeks neuronu ukrytego	$j = 1, \ldots, L$ (na W3 L to liczba neuronów ukrytych, na W4 w zapisie macierzowym L to numer ostatniej warstwy – uwaga na to)
k	indeks neuronu wyjściowego	$k = 1, \ldots, M$; M = liczba klas
górny indeks ^{1}, ^{2}	„w warstwie 1 / 2”	**numer warstwy, nie potęga**. w^2_{kj} to waga w warstwie 2, nie „w do kwadratu”. Potęgę wykład pisze tylko przy nawiasach, np. $(\delta_{pk})^2$.
w^1_{ji}	„wu jeden jot i”	waga **do** neuronu ukrytego j **od** wejścia i. Kolejność: najpierw dokąd, potem skąd.
w^2_{kj}	„wu dwa ka jot”	waga do neuronu wyjściowego k od neuronu ukrytego j
z^1_{pj}	„całkowite pobudzenie”	suma ważona wejść neuronu ukrytego j (w ćw. 3 nazywaliśmy to z albo „wyjście warstwy liniowej”)
f^1_j, f^2_k	„funkcja aktywacji”	funkcja, którą neuron nakłada na z: sigmoida, ReLU, softmax… Górny indeks – w której warstwie.
$a^1_{pj} = f^1_j(z^1_{pj})$	„aktywacja”, „wyjście neuronu”	to, co neuron ukryty j wysyła dalej
$\hat{y}_{pk} = f^2_k(z^2_{pk})$	„igrek z daszkiem”	odpowiedź sieci na wyjściu k. Daszek = „to, co sieć przewidziała”.
y_{pk}	„igrek”	prawdziwa (żądana) odpowiedź na wyjściu k
δ_{pk}	„delta”, „błąd neuronu”	na W3 i W4 (wersja dla pojedynczych wag): $\delta^2_{pk} = y_{pk} - \hat{y}_{pk}$, czyli **prawda minus odpowiedź**. Patrz ramka „uwaga” niżej.
μ	„mi”	współczynnik uczenia (w ćw. 3: α, `lr`). Dodatni, mniejszy od 1.

t, $t+1$	„chwila t”	numer kroku uczenia. $w(t+1)$ = waga po kroku, $w(t)$ = przed krokiem.
$\Delta_p w$	„delta pe wu”	o ile zmieniamy wagę po wzorcu p (zanim pomnożymy przez μ)
$\mathcal{L}$, L, E_p, C	„strata”, „błąd”, „koszt”	to samo: liczba mówiąca, jak bardzo sieć się myli. W3 pisze $\mathcal{L}$, W4 przy MSE pisze E_p.
$\sum_{k=1}^{M}$	„suma po k od 1 do M”	dodaj wyrażenie dla $k = 1, 2, \ldots, M$
$\odot$	„iloczyn Hadamarda”	mnożenie wektorów element po elemencie: $[1,2] \odot [3,4] = [3,8]$

Uwaga na znak delty (najczęstsza pułapka):

1. W3 i W4 (wersja dla pojedynczych wag): $\delta = y - \hat{y}$ (prawda minus odpowiedź), a krok to **plus**: $w(t+1) = w(t) + \mu\,\delta\,a$.
2. W4 (wersja macierzowa, slajd 19) i ćw. 3: $\delta = \frac{\partial L}{\partial z} = \hat{y} - y$ (odpowiedź minus prawda), a krok to **minus**: $W \leftarrow W - \alpha\,\delta\,a^T$.
3. To jest **ten sam krok**: $+\mu(y-\hat{y})a = -\mu(\hat{y}-y)a$. Zmienił się tylko znak w definicji delty, więc zmienia się znak przed nią.

Uwaga na bias: wykład pisze $z^2_{pk} = \sum_{j=0}^{L} w^2_{kj} a^1_{pj}$, z sumą **od** $j = 0$. To sztuczka: dokłada się udawany neuron nr 0, który zawsze daje $a^1_{p0} = 1$. Wtedy $w^2_{k0} \cdot 1$ to po prostu bias b^2_k. Czyli $\sum_{j=0}^{L} w_{kj} a_j = b_k + \sum_{j=1}^{L} w_{kj} a_j$.

1. Przejście w przód: przykład ze slajdów przeliczony

1.1. Neuron (przypomnienie)

$$z^1_{pj} = \sum_{i=1}^{N} w^1_{ji}\, x_{pi} + b^1_j, \qquad a^1_{pj} = f^1_j(z^1_{pj})$$

Po ludzku: neuron mnoży każde wejście przez swoją wagę, dodaje wszystko i bias, a potem przepuszcza wynik przez funkcję aktywacji. Warstwa to kilka takich neuronów obok siebie, każdy z własnymi wagami. Sieć to kilka warstw jedna za drugą: wyjścia jednej są wejściami następnej.

1.2. Sieć ze slajdu W3: 30

Wszystkie neurony mają sigmoidę $\sigma(z) = \frac{1}{1+e^{-z}}$. Dane (w macierzy wiersz j = wagi neuronu j):

warstwa	wagi W	bias b
1	$\begin{bmatrix} 1 & -2 \\ -1 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$
2	$\begin{bmatrix} 2 & -1 \\ -2 & -1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$
3	$\begin{bmatrix} 3 & -1 \\ -1 & 4 \end{bmatrix}$	$\begin{bmatrix} -2 \\ 2 \end{bmatrix}$

Na liczbach, wejście $x = [1, -1]$:

1. warstwa 1, neuron 1: $z = 1 \cdot 1 + (-2) \cdot (-1) + 1 = 1 + 2 + 1 = 4$; $\sigma(4) = \frac{1}{1+e^{-4}} = \frac{1}{1+0{,}0183} = 0{,}98$
2. warstwa 1, neuron 2: $z = (-1) \cdot 1 + 1 \cdot (-1) + 0 = -2$; $\sigma(-2) = \frac{1}{1+e^{2}} = \frac{1}{1+7{,}389} = 0{,}12$
3. warstwa 2, neuron 1: $z = 2 \cdot 0{,}98 + (-1) \cdot 0{,}12 = 1{,}96 - 0{,}12 = 1{,}84$; $\sigma(1{,}84) = 0{,}86$
4. warstwa 2, neuron 2: $z = (-2) \cdot 0{,}98 + (-1) \cdot 0{,}12 = -1{,}96 - 0{,}12 = -2{,}08$; $\sigma(-2{,}08) = 0{,}11$
5. warstwa 3, neuron 1: $z = 3 \cdot 0{,}86 + (-1) \cdot 0{,}11 + (-2) = 2{,}59 - 0{,}11 - 2 = 0{,}48$; $\sigma(0{,}48) = 0{,}62$
6. warstwa 3, neuron 2: $z = (-1) \cdot 0{,}86 + 4 \cdot 0{,}11 + 2 = -0{,}86 + 0{,}44 + 2 = 1{,}58$; $\sigma(1{,}58) = 0{,}83$
7. wynik: $f([1, -1]) = [0{,}62;\ 0{,}83]$ – tak jak na slajdzie W3: 33.

1.3. To samo macierzowo (slajd W3: 32)

Na liczbach, warstwa 1:

1. $\begin{bmatrix} 1 & -2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + (-2)(-1) \\ (-1) \cdot 1 + 1 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$ (wiersz razy kolumna)
2. plus bias: $\begin{bmatrix} 3 \\ -2 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 4 \\ -2 \end{bmatrix}$
3. sigmoida na każdym elemencie osobno: $\sigma\left(\begin{bmatrix} 4 \\ -2 \end{bmatrix}\right) = \begin{bmatrix} 0{,}98 \\ 0{,}12 \end{bmatrix}$

$$\mathbf{y} = f(\mathbf{x}) = \sigma\Big(W^L \cdots \sigma\big(W^2\, \sigma(W^1 \mathbf{x} + \mathbf{b}^1) + \mathbf{b}^2\big) \cdots + \mathbf{b}^L\Big)$$

Po ludzku (W3: 33–35): cała sieć to jedna funkcja złożona z warstw. Różne wagi i biasy dają różne funkcje: ta sama sieć dla wejścia $[0, 0]$ daje $[0{,}51;\ 0{,}85]$. Zapis macierzowy jest lepszy, bo mnożenie macierzy robią szybko gotowe biblioteki i karty graficzne (GPU).

Sprawdź się 1: policz wyjście tej samej sieci dla $x = [0, 0]$ i sprawdź, czy wychodzi $[0{,}51;\ 0{,}85]$.

2. Wiele klas: one-hot, softmax, entropia krzyżowa

2.1. Kodowanie „1 z n” (one-hot), slajd W3: 3

Sieć ma rozpoznać: kot, pies, wielbłąd, kapelusz, kwiat. Kolejność klas ustalamy raz: $[\text{kot}, \text{pies}, \text{wielbłąd}, \text{kapelusz}, \text{kwiat}]$. Żądane wyjście:

klasa	**wektor** y
kot	$[1,0,0,0,0]^T$
pies	$[0,1,0,0,0]^T$
wielbłąd	$[0,0,1,0,0]^T$
kapelusz	$[0,0,0,1,0]^T$
kwiat	$[0,0,0,0,1]^T$

Po ludzku: tyle wyjść, ile klas. W wektorze jest jedna jedynka – w miejscu prawdziwej klasy – i same zera. Ważna konsekwencja na później: **suma wszystkich y_k wynosi zawsze 1**.

2.2. Softmax, slajd W3: 10

Problem: wyjścia neuronów z_k mogą być dowolnymi liczbami (np. 3, 1, -3) i nie da się ich czytać jako prawdopodobieństw. Sigmoida na każdym wyjściu osobno też nie pomaga: każde wyjście jest w $(0,1)$, ale razem nie sumują się do 1. Softmax naprawia jedno i drugie.

$$\hat{y}_k = \text{softmax}_k(\mathbf{z}) = \frac{e^{z_k}}{\sum_{j=1}^{M} e^{z_j}}$$

Co oznacza każdy symbol: z_k – pobudzenie neuronu wyjściowego k; e^{z_k} – liczba $e \approx 2{,}718$ do potęgi z_k (zawsze dodatnia); mianownik – suma e^z ze **wszystkich** wyjść; $\hat{y}_k$ – prawdopodobieństwo klasy k według sieci.

Po ludzku: (1) e^z robi z każdej liczby liczbę dodatnią, a większe z dostaje dużo większy kawałek. (2) Dzielenie przez sumę sprawia, że wszystko sumuje się do 1. Wynik: $0 < \hat{y}_k < 1$ i $\sum_k \hat{y}_k = 1$. Predykcja (którą klasę wybiera sieć) = numer największego $\hat{y}_k$, czyli $y_{\text{pred}} = \arg\max_k \hat{y}_k$.

Na liczbach (slajd W3: 10), $z = [3, 1, -3]$**:**

1. potęgi: $e^3 = 20{,}09$, $e^1 = 2{,}72$, $e^{-3} = 0{,}05$
2. suma: $20{,}09 + 2{,}72 + 0{,}05 = 22{,}85$
3. dzielę: $\hat{y}_1 = \frac{20{,}09}{22{,}85} = 0{,}88$, $\hat{y}_2 = \frac{2{,}72}{22{,}85} = 0{,}12$, $\hat{y}_3 = \frac{0{,}05}{22{,}85} = 0{,}002 \approx 0$
4. sprawdzenie: $0{,}88 + 0{,}12 + 0{,}002 = 1$ ✓; predykcja: klasa 1

2.3. Entropia krzyżowa dla wielu klas, slajd W3: 5

$$\mathcal{L}_{CE} = -\sum_{k=1}^{M} y_k \ln \hat{y}_k$$

Co oznacza każdy symbol: $y_k \in \{0, 1\}$ – prawda w kodowaniu one-hot; $\hat{y}_k \in (0, 1)$ – odpowiedź sieci (z softmaxa); $\ln$ – logarytm naturalny; M – liczba klas.

Po ludzku: w sumie tylko jeden składnik jest niezerowy – ten, gdzie $y_k = 1$ (prawdziwa klasa). Reszta jest mnożona przez 0. Więc $\mathcal{L} = -\ln(\hat{y}_{\text{prawdziwej klasy}})$. Kara zależy tylko od tego, ile prawdopodobieństwa sieć dała **prawdziwej** klasie.

Na liczbach, $\hat{y} = [0{,}88;\ 0{,}12;\ 0{,}002]$**:**

1. prawdziwa klasa 1, $y = [1, 0, 0]$: $\mathcal{L} = -(1 \cdot \ln 0{,}88 + 0 + 0) = -(-0{,}128) = 0{,}128$ (mała kara)
2. prawdziwa klasa 2, $y = [0, 1, 0]$: $\mathcal{L} = -\ln 0{,}12 = 2{,}12$
3. prawdziwa klasa 3, $y = [0, 0, 1]$: $\mathcal{L} = -\ln 0{,}002 = 6{,}2$ (ogromna kara: sieć była pewna i się myliła)

Odpowiedź na pytania ze slajdu W3: 5 („Jakie to ma konsekwencje? Czy można użyć MSE?”): konsekwencja one-hot: liczy się tylko wyjście prawdziwej klasy, a $\sum_k y_k = 1$, co bardzo upraszcza pochodną (rozdz. 3). MSE można użyć (W4 robi to w rozdz. 6), ale z sigmoidą na wyjściu uczy się wolniej, bo w gradiencie zostaje pochodna sigmoidy, która w nasyceniu jest prawie zerem.

3. Warstwa wyjściowa: skąd $\hat{y}_k - y_k$

3.0. Dlaczego „wstecz”? (najpierw intuicja)

Po ludzku: nic nie płynie w tył. „Wstecz” mówi tylko, **w jakiej kolejności liczymy pochodne**. Błąd znamy tylko na wyjściu (tylko tam mamy prawdę y). Żeby wiedzieć, jak waga na początku sieci wpływa na stratę, trzeba znać wszystko, co jest za nią. Dlatego zaczynamy od wyjścia i „winę” przekazujemy warstwa po warstwie w stronę wejścia. Każdy kawałek liczymy raz i używamy dalej.

Na liczbach (mini-sieć bez aktywacji): $x = 2$, $w_1 = 3$, $w_2 = 4$, prawda $y = 20$, strata $L = \frac{1}{2}(\hat{y} - y)^2$.

1. w przód: $a = w_1 x = 3 \cdot 2 = 6$; $\hat{y} = w_2 a = 4 \cdot 6 = 24$; $L = \frac{1}{2}(24 - 20)^2 = 8$
2. wstecz, wyjście: $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 24 - 20 = 4$ (tu „widać” błąd)
3. waga w_2: $\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot a = 4 \cdot 6 = 24$
4. neuron ukryty: $\frac{\partial L}{\partial a} = \frac{\partial L}{\partial \hat{y}} \cdot w_2 = 4 \cdot 4 = 16$ („wina” neuronu ukrytego, policzona z winy wyjścia)
5. waga w_1: $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot x = 16 \cdot 2 = 32$
6. sprawdzenie wprost: $L = \frac{1}{2}(w_2 w_1 x - y)^2$, więc $\frac{\partial L}{\partial w_1} = (\hat{y} - y)\, w_2\, x = 4 \cdot 4 \cdot 2 = 32$ ✓
7. krok 5 potrzebował wyniku kroku 4, a krok 4 wyniku kroku 2. Stąd kolejność: od wyjścia do wejścia. Przejście w przód musi być pierwsze, bo pochodne używają wartości ($a = 6$, $\hat{y} = 24$).

W prawdziwej sieci jest dokładnie to samo, tylko „wina” neuronu to δ, a zamiast jednej wagi w_2 jest suma po wszystkich wyjściach. Rozdz. 3 liczy δ na wyjściu, rozdz. 4 przekazuje ją do warstwy ukrytej.

3.1. Co chcemy policzyć

$$w_{kj}^2(t+1) = w_{kj}^2(t) + \mu\, \Delta_p w_{kj}^2(t), \qquad \Delta_p w_{kj}^2 = -\frac{\partial \mathcal{L}_p}{\partial w_{kj}^2}$$

Po ludzku: każdą wagę przesuwamy w kierunku przeciwnym do gradientu (tam, gdzie strata maleje), o krok proporcjonalny do gradientu. To jest zwykły spadek gradientu z ćw. 2, tylko dla jednej wagi naraz. Cała praca to policzyć $\frac{\partial \mathcal{L}}{\partial w_{kj}^2}$.

3.2. Reguła łańcuchowa dla wagi w_{kj}^2 (slajd W3: 8)

Waga w_{kj}^2 wpływa na stratę po drodze: $w_{kj}^2 \to z_k^2 \to \hat{y} \to \mathcal{L}$. Pochodna drogi = iloczyn pochodnych ogniw:

$$\frac{\partial \mathcal{L}}{\partial w_{kj}^2} = \underbrace{\frac{\partial \mathcal{L}}{\partial z_k^2}}_{\text{to policzymy w 3.3–3.5}} \cdot \underbrace{\frac{\partial z_k^2}{\partial w_{kj}^2}}_{= a_j^1}$$

Ostatnie ogniwo jest łatwe:

1. $z_k^2 = w_{k0}^2 a_0^1 + w_{k1}^2 a_1^1 + \cdots + w_{kj}^2 a_j^1 + \cdots + w_{kL}^2 a_L^1$
2. różniczkujemy po w_{kj}^2: wszystkie inne składniki nie zawierają w_{kj}^2, więc ich pochodna to 0 („po tym różniczkujemy, więc pozostałe składowe nie wchodzą do pochodnej”)
3. zostaje $w_{kj}^2 a_j^1$, pochodna po w_{kj}^2 to a_j^1 (jak $3x \to 3$)

3.3. Dwie pochodne pomocnicze

1. pochodna potęgi e: $\frac{d}{dz} e^z = e^z$ (funkcja wykładnicza jest swoją własną pochodną)
2. pochodna ilorazu: $\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$ (pochodna góry razy dół, minus góra razy pochodna dołu, przez dół do kwadratu)
3. pochodna straty po wyjściu $\hat{y}_j$ (slajd W3: 9): $\mathcal{L} = -\sum_m y_m \ln \hat{y}_m$; po $\hat{y}_j$ zostaje tylko składnik $m = j$, a $(\ln u)' = \frac{1}{u}$, więc $\frac{\partial \mathcal{L}}{\partial \hat{y}_j} = -\frac{y_j}{\hat{y}_j}$

3.4. Pochodna softmaxa (slajdy W3: 11–12)

Oznaczam $S = \sum_m e^{z_m}$ (suma w mianowniku). Wtedy $\hat{y}_j = \frac{e^{z_j}}{S}$. Pochodna S po z_k: w sumie tylko e^{z_k} zależy od z_k, więc $\frac{\partial S}{\partial z_k} = e^{z_k}$. Każde $\hat{y}_j$ zależy od **każdego** z_k (przez mianownik), więc są dwa przypadki.

Przypadek $j = k$ (to samo wyjście): $u = e^{z_k}$, $u' = e^{z_k}$, $v = S$, $v' = e^{z_k}$.

1. $\frac{\partial \hat{y}_k}{\partial z_k} = \frac{e^{z_k} \cdot S - e^{z_k} \cdot e^{z_k}}{S^2}$ (wzór na iloraz)
2. rozbijam na dwa ułamki: $= \frac{e^{z_k}}{S} \cdot \frac{S}{S} - \frac{e^{z_k}}{S} \cdot \frac{e^{z_k}}{S} = \frac{e^{z_k}}{S}\left(1 - \frac{e^{z_k}}{S}\right)$
3. $\frac{e^{z_k}}{S}$ to $\hat{y}_k$, więc: $\frac{\partial \hat{y}_k}{\partial z_k} = \hat{y}_k(1 - \hat{y}_k)$ (taki sam kształt jak pochodna sigmoidy)

Przypadek $j \neq k$ (inne wyjście): $u = e^{z_j}$ nie zależy od z_k, więc $u' = 0$; $v = S$, $v' = e^{z_k}$.

4. $\frac{\partial \hat{y}_j}{\partial z_k} = \frac{0 \cdot S - e^{z_j} \cdot e^{z_k}}{S^2} = -\frac{e^{z_j}}{S} \cdot \frac{e^{z_k}}{S} = -\hat{y}_j \hat{y}_k$

> **Po ludzku:** gdy podniosę z_k, to $\hat{y}_k$ rośnie (pochodna dodatnia), a wszystkie inne $\hat{y}_j$ maleją (pochodna ujemna), bo suma musi zostać równa 1.

3.5. Składamy (slajd W3: 13)

z_k wpływa na stratę przez **wszystkie** wyjścia $\hat{y}_j$, więc sumuję po drogach (reguła łańcuchowa dla wielu dróg):

1. $\frac{\partial \mathcal{L}}{\partial z_k} = \sum_j \frac{\partial \mathcal{L}}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_k} = \underbrace{\frac{\partial \mathcal{L}}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_k}}_{j=k} + \sum_{j \neq k} \frac{\partial \mathcal{L}}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_k}$
2. wstawiam z 3.3 i 3.4: $= \left(-\frac{y_k}{\hat{y}_k}\right) \cdot \hat{y}_k(1 - \hat{y}_k) + \sum_{j \neq k} \left(-\frac{y_j}{\hat{y}_j}\right) \cdot (-\hat{y}_j \hat{y}_k)$
3. skracam $\hat{y}_k$ w pierwszym i $\hat{y}_j$ w drugim: $= -y_k(1 - \hat{y}_k) + \sum_{j \neq k} y_j \hat{y}_k$
4. wymnażam nawias: $= -y_k + y_k \hat{y}_k + \hat{y}_k \sum_{j \neq k} y_j$
5. wyciągam $\hat{y}_k$: $= -y_k + \hat{y}_k\left(y_k + \sum_{j \neq k} y_j\right) = -y_k + \hat{y}_k \sum_j y_j$
6. one-hot: $\sum_j y_j = 1$ („mamy kodowanie 1 z n, więc to wyrażenie jest zawsze = 1”): $\frac{\partial \mathcal{L}}{\partial z_k} = \hat{y}_k - y_k$

$$\frac{\partial \mathcal{L}}{\partial w^2_{kj}} = (\hat{y}_{pk} - y_{pk})\, a^1_{pj}, \qquad \delta^2_{pk} = y_{pk} - \hat{y}_{pk}, \qquad w^2_{kj}(t+1) = w^2_{kj}(t) + \mu\, \delta^2_{pk}\, a^1_{pj}$$

Co oznacza każdy symbol: δ^2_{pk} – błąd neuronu wyjściowego k dla wzorca p (prawda minus odpowiedź); a^1_{pj} – to, co do neuronu k przyszło z neuronu ukrytego j (dla biasu: $a^1_{p0} = 1$); μ – współczynnik uczenia.

Po ludzku: zmiana wagi = μ · (o ile sieć się pomyliła na wyjściu k) · (jak mocny sygnał szedł tą wagą). Gdy sieć dała za mało ($y > \hat{y}$), $\delta > 0$ i wagi od aktywnych neuronów rosną. Gdy neuron ukryty milczał ($a_j = 0$), jego waga się nie zmienia – nie miał wpływu na błąd. To ten sam wynik co $\delta = p - y$ w ćw. 3, tylko teraz dla wielu klas.

Na liczbach, sprawdzenie wzoru 3.5 bez skrótu: $\hat{y} = [0{,}9807;\ 0{,}0193]$, $y = [0, 1]$, liczę $\frac{\partial \mathcal{L}}{\partial z_1}$.

1. $\mathcal{L} = -\ln \hat{y}_2$, więc tylko droga przez $\hat{y}_2$: $\frac{\partial \mathcal{L}}{\partial \hat{y}_2} = -\frac{1}{0{,}0193} = -51{,}8$
2. $j = 2 \neq k = 1$: $\frac{\partial \hat{y}_2}{\partial z_1} = -\hat{y}_2 \hat{y}_1 = -0{,}0193 \cdot 0{,}9807 = -0{,}01893$
3. iloczyn: $(-51{,}8) \cdot (-0{,}01893) = 0{,}9807$
4. skrót: $\hat{y}_1 - y_1 = 0{,}9807 - 0 = 0{,}9807$ ✓

4. Warstwa ukryta: skąd δ_j^1 (slajdy W3: 16–19)

Problem ze slajdu W3: 2: „nie wiemy, ile wynosi błąd w warstwie ukrytej”. Dla neuronu wyjściowego znamy prawdę y_k, więc błąd to $y_k - \hat{y}_k$. Dla neuronu ukrytego nikt nie mówi, jaka powinna być jego wartość. Rozwiązanie: policzyć, jak bardzo ten neuron przyczynił się do błędów na wyjściu.

4.1. Droga od wagi ukrytej do straty

Waga w_{ji}^1 (wejście i → neuron ukryty j) wpływa na stratę tak: $w_{ji}^1 \to z_j^1 \to a_j^1 \to$ **wszystkie** z_k^2 (bo neuron j jest połączony z każdym wyjściem) $\to \mathcal{L}$. Stąd suma po k:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^1} = \sum_{k=1}^{M} \underbrace{\frac{\partial \mathcal{L}}{\partial z_k^2}}_{\hat{y}_k - y_k} \cdot \underbrace{\frac{\partial z_k^2}{\partial a_j^1}}_{w_{kj}^2} \cdot \underbrace{\frac{\partial a_j^1}{\partial z_j^1}}_{f_j'^1(z_j^1)} \cdot \underbrace{\frac{\partial z_j^1}{\partial w_{ji}^1}}_{x_i}$$

Każde ogniwo osobno (slajd W3: 17):

1. $\frac{\partial \mathcal{L}}{\partial z_k^2} = \hat{y}_k - y_k$ – to już mamy z rozdz. 3
2. $z_k^2 = \sum_j w_{kj}^2 a_j^1$; po a_j^1 zostaje tylko składnik z a_j^1, więc $\frac{\partial z_k^2}{\partial a_j^1} = w_{kj}^2$
3. $a_j^1 = f_j^1(z_j^1)$, więc $\frac{\partial a_j^1}{\partial z_j^1} = f_j'^1(z_j^1)$ – pochodna funkcji aktywacji warstwy ukrytej (dla sigmoidy: $a_j^1(1 - a_j^1)$, dla ReLU: 1 albo 0)
4. $z_j^1 = \sum_i w_{ji}^1 x_i$; po w_{ji}^1 zostaje x_i, więc $\frac{\partial z_j^1}{\partial w_{ji}^1} = x_i$
5. ogniwa 3 i 4 nie zależą od k, więc wyciągam je przed sumę: $\frac{\partial \mathcal{L}}{\partial w_{ji}^1} = f_j'^1(z_j^1)\, x_i \sum_{k=1}^{M} (\hat{y}_k - y_k)\, w_{kj}^2$
6. przyrost to minus gradient, a $-(\hat{y}_k - y_k) = y_k - \hat{y}_k = \delta_k^2$: $\Delta_p w_{ji}^1 = f_j'^1(z_{pj}^1)\, x_{pi} \sum_{k=1}^{M} \delta_{pk}^2\, w_{kj}^2$

$$\delta_{pj}^1 = f_j'^1(z_{pj}^1) \sum_{k=1}^{M} \delta_{pk}^2\, w_{kj}^2, \qquad w_{ji}^1(t+1) = w_{ji}^1(t) + \mu\, \delta_{pj}^1\, x_{pi}$$

Co oznacza każdy symbol: δ_{pj}^1 – błąd neuronu ukrytego j; $f_j'^1(z_{pj}^1)$ – nachylenie jego funkcji aktywacji w punkcie, w którym teraz pracuje; $\sum_k \delta_{pk}^2 w_{kj}^2$ – błędy wyjść przesłane wstecz po tych samych wagach, po których sygnał szedł w przód; x_{pi} – wejście i.

Po ludzku: błąd neuronu ukrytego = suma błędów wyjść, każdy pomnożony przez wagę łączącą ten neuron z danym wyjściem (im mocniejsze połączenie, tym większa „wina”), razy nachylenie jego aktywacji. Wzór na zmianę wagi wygląda **tak samo** jak w warstwie wyjściowej: $\mu \cdot \delta \cdot$ (wejście tej wagi). Różni się tylko sposób liczenia δ. Bo we wzorze jest f', funkcja aktywacji musi być **różniczkowalna** (W3: 18).

Ćw. 3 kontra wykład: w ćw. 3 to samo było zapisane jako $dx = W^T \delta$ (przesłanie przez warstwę liniową) i potem $\odot$ z pochodną ReLU. $\sum_k \delta_k w_{kj}$ to dokładnie element j wektora $W^T \delta$ (rozdz. 7).

5. Pełny przykład na liczbach (softmax + entropia krzyżowa)

Dane. Sieć 2 → 2 → 2. Warstwa ukryta: sigmoida, wagi i bias z przykładu ze slajdu W3: 30. Warstwa wyjściowa: softmax.

co	wartość
wejście x	$[1, -1]$
prawda y (klasa 2 z dwóch)	$[0, 1]$
W^1, b^1	$w^1_{11}=1$, $w^1_{12}=-2$, $w^1_{21}=-1$, $w^1_{22}=1$; $b^1=[1,0]$
W^2, b^2	$w^2_{11}=2$, $w^2_{12}=-1$, $w^2_{21}=-2$, $w^2_{22}=-1$; $b^2=[0,0]$
μ	$0{,}5$

5.1. Przejście w przód

1. $z^1_1 = 1\cdot 1 + (-2)(-1) + 1 = 4$; $z^1_2 = (-1)\cdot 1 + 1\cdot(-1) + 0 = -2$
2. $a^1_1 = \sigma(4) = 0{,}9820$; $a^1_2 = \sigma(-2) = 0{,}1192$
3. $z^2_1 = 2\cdot 0{,}9820 + (-1)\cdot 0{,}1192 + 0 = 1{,}9640 - 0{,}1192 = 1{,}8448$
4. $z^2_2 = (-2)\cdot 0{,}9820 + (-1)\cdot 0{,}1192 + 0 = -1{,}9640 - 0{,}1192 = -2{,}0832$
5. softmax: $e^{1{,}8448} = 6{,}327$; $e^{-2{,}0832} = 0{,}1245$; suma $= 6{,}4515$
6. $\hat{y}_1 = \frac{6{,}327}{6{,}4515} = 0{,}9807$; $\hat{y}_2 = \frac{0{,}1245}{6{,}4515} = 0{,}0193$
7. strata: prawdziwa jest klasa 2, więc $\mathcal{L} = -\ln \hat{y}_2 = -\ln 0{,}0193 = 3{,}948$ (sieć jest pewna, że to klasa 1, i się myli)

5.2. Błędy wyjść δ^2

1. $\delta^2_1 = y_1 - \hat{y}_1 = 0 - 0{,}9807 = -0{,}9807$ (wyjście 1 za duże → ujemny błąd)
2. $\delta^2_2 = y_2 - \hat{y}_2 = 1 - 0{,}0193 = 0{,}9807$ (wyjście 2 za małe → dodatni błąd)

5.3. Błędy neuronów ukrytych δ^1

1. pochodna sigmoidy $a(1-a)$: $f'(z^1_1) = 0{,}9820\cdot 0{,}0180 = 0{,}01766$; $f'(z^1_2) = 0{,}1192\cdot 0{,}8808 = 0{,}1050$
2. neuron ukryty 1 łączy się z wyjściem 1 wagą $w^2_{11}=2$ i z wyjściem 2 wagą $w^2_{21}=-2$: $\sum_k \delta^2_k w^2_{k1} = (-0{,}9807)\cdot 2 + 0{,}9807\cdot(-2) = -1{,}9614 - 1{,}9614 = -3{,}9228$
3. neuron ukryty 2: wagi $w^2_{12}=-1$, $w^2_{22}=-1$: $\sum_k \delta^2_k w^2_{k2} = (-0{,}9807)(-1) + 0{,}9807\cdot(-1) = 0{,}9807 - 0{,}9807 = 0$
4. $\delta^1_1 = 0{,}01766\cdot(-3{,}9228) = -0{,}0693$; $\delta^1_2 = 0{,}1050\cdot 0 = 0$
5. zauważ: δ^1_1 jest mała, bo neuron 1 pracuje w nasyceniu ($z=4$, $f'=0{,}018$). To jest zanikający gradient w małej skali (rozdz. 10).

5.4. Nowe wagi ($+\mu \cdot \delta \cdot$ wejście wagi)

1. w_{11}^2: $2 + 0{,}5 \cdot (-0{,}9807) \cdot 0{,}9820 = 2 - 0{,}4815 = 1{,}5185$
2. w_{12}^2: $-1 + 0{,}5 \cdot (-0{,}9807) \cdot 0{,}1192 = -1 - 0{,}0585 = -1{,}0585$
3. w_{21}^2: $-2 + 0{,}5 \cdot 0{,}9807 \cdot 0{,}9820 = -2 + 0{,}4815 = -1{,}5185$
4. w_{22}^2: $-1 + 0{,}5 \cdot 0{,}9807 \cdot 0{,}1192 = -1 + 0{,}0585 = -0{,}9415$
5. biasy wyjścia (wejście biasu = 1): $b_1^2 = 0 + 0{,}5 \cdot (-0{,}9807) = -0{,}4903$; $b_2^2 = 0 + 0{,}5 \cdot 0{,}9807 = 0{,}4903$
6. w_{11}^1: $1 + 0{,}5 \cdot (-0{,}0693) \cdot 1 = 1 - 0{,}0346 = 0{,}9654$; w_{12}^1: $-2 + 0{,}5 \cdot (-0{,}0693) \cdot (-1) = -2 + 0{,}0346 = -1{,}9654$
7. $b_1^1 = 1 + 0{,}5 \cdot (-0{,}0693) = 0{,}9654$; wagi neuronu ukrytego 2 bez zmian ($\delta_2^1 = 0$)
8. kolejność aktualizacji w jednej warstwie nie ma znaczenia (W3: 24), ale wszystkie δ liczymy na **starych** wagach

5.5. Sprawdzenie

1. ta sama sieć z nowymi wagami daje $\hat{y} = [0{,}8789;\ 0{,}1211]$
2. strata: $-\ln 0{,}1211 = 2{,}111$ (było $3{,}948$) – spadła po jednym kroku ✓
3. sprawdzenie gradientu numerycznie (rozdz. C w NAUKA_cw3): ruszam w_{11}^2 o $h = 10^{-6}$ i liczę $\frac{\mathcal{L}(w+h) - \mathcal{L}(w)}{h} = 0{,}9631$; wzór: $(\hat{y}_1 - y_1)\, a_1^1 = 0{,}9807 \cdot 0{,}9820 = 0{,}9631$ ✓

Sprawdź się 5: ta sama sieć, ale prawdą jest klasa 1: $y = [1, 0]$. Policz $\mathcal{L}$, δ_1^2, δ_2^2, δ_1^1 i nową wagę w_{11}^2. Dlaczego zmiany są tak małe?

6. Wersja z MSE (wykład 4, slajdy 4–14)

> *„W warstwie wyjściowej – sigmoidalna funkcja aktywacji (nie używamy softmax). Będziemy minimalizować sumę kwadratów tych błędów po wszystkich wyjściach.”*

$$E_p = \frac{1}{2}\sum_{k=1}^{M}(y_{pk} - \hat{y}_{pk})^2, \qquad \hat{y}_{pk} = f_k^2(z_{pk}^2) = \sigma(z_{pk}^2)$$

Co oznacza każdy symbol: E_p – błąd dla wzorca p; $(y - \hat{y})^2$ – kwadrat pomyłki na wyjściu k; $\frac{1}{2}$ – stała dla wygody: skróci się z 2 z pochodnej kwadratu.

6.1. Warstwa wyjściowa (slajdy W4: 6–8)

Łańcuch: $w_{kj}^2 \to z_k^2 \to \hat{y}_k \to E$. Teraz $\hat{y}_k$ zależy tylko od swojego z_k^2 (sigmoida na każdym wyjściu osobno), więc nie ma sumy po drogach jak przy softmaxie.

1. $\frac{\partial E}{\partial \hat{y}_k}$: z sumy zostaje tylko składnik k; $\frac{1}{2}\cdot 2\cdot(y_k - \hat{y}_k)\cdot(-1) = -(y_k - \hat{y}_k)$ (minus z pochodnej wnętrza: $\hat{y}$ jest z minusem)
2. $\frac{\partial \hat{y}_k}{\partial z_k^2} = f_k'^2(z_k^2)$ (dla sigmoidy: $\hat{y}_k(1-\hat{y}_k)$)
3. $\frac{\partial z_k^2}{\partial w_{kj}^2} = a_j^1$ (jak w 3.2)
4. iloczyn: $\frac{\partial E}{\partial w_{kj}^2} = -(y_k - \hat{y}_k)\, f_k'^2(z_k^2)\, a_j^1$

$$\delta_{pk}^2 = (y_{pk} - \hat{y}_{pk})\, f_k'^2(z_{pk}^2), \qquad w_{kj}^2(t+1) = w_{kj}^2(t) + \mu\, \delta_{pk}^2\, a_{pj}^1$$

Po ludzku (W4: 8 „wzór identyczny jak dla L_{CE}, ale definicja błędu się różni”): krok wygląda tak samo. Jedyna różnica: w MSE delta ma dodatkowy czynnik $f'(z)$, pochodną aktywacji wyjścia. Przy entropii krzyżowej ten czynnik się skrócił (rozdz. 3.5 i ćw. 3: $\delta = p - y$).

6.2. Warstwa ukryta (slajdy W4: 9–12)

1. rozpisuję E tak, żeby było widać wagi ukryte: $E = \frac{1}{2}\sum_k \left(y_k - f_k^2\left(\sum_j w_{kj}^2\, f_j^1(\sum_i w_{ji}^1 x_i)\right)\right)^2$ – „tutaj ukryte są wagi z warstwy ukrytej”
2. reguła łańcuchowa przez każde wyjście k: $\frac{\partial E}{\partial w_{ji}^1} = -\sum_{k=1}^{M}(y_k - \hat{y}_k)\cdot f_k'^2(z_k^2)\cdot w_{kj}^2 \cdot f_j'^1(z_j^1)\cdot x_i$
3. pierwsze dwa czynniki to δ_k^2 z 6.1: $\Delta_p w_{ji}^1 = f_j'^1(z_j^1)\, x_i \sum_k \delta_k^2\, w_{kj}^2$

$$\delta_{pj}^1 = f_j'^1(z_{pj}^1)\sum_{k=1}^{M}\delta_{pk}^2\, w_{kj}^2, \qquad w_{ji}^1(t+1) = w_{ji}^1(t) + \mu\, \delta_{pj}^1\, x_{pi}$$

Po ludzku: wzór na warstwę ukrytą jest **identyczny** jak przy entropii krzyżowej. Funkcja straty zmienia tylko to, jak liczymy δ na wyjściu. Reszta algorytmu (przesłanie wstecz) jest zawsze taka sama.

6.3. Na liczbach: ta sama sieć co w rozdz. 5, ale wyjście z sigmoidą i MSE

Dane: $x = [1, -1]$, $y = [0, 1]$, $\mu = 0{,}5$, wagi jak w rozdz. 5, więc $a^1 = [0{,}9820;\ 0{,}1192]$ i $z^2 = [1{,}8448;\ -2{,}0832]$.

1. wyjście: $\hat{y}_1 = \sigma(1{,}8448) = 0{,}8635$; $\hat{y}_2 = \sigma(-2{,}0832) = 0{,}1107$
2. błąd: $E = \frac{1}{2}[(0 - 0{,}8635)^2 + (1 - 0{,}1107)^2] = \frac{1}{2}[0{,}7456 + 0{,}7909] = 0{,}7682$
3. pochodne wyjścia: $f'(z_1^2) = 0{,}8635 \cdot 0{,}1365 = 0{,}1179$; $f'(z_2^2) = 0{,}1107 \cdot 0{,}8893 = 0{,}0985$
4. $\delta_1^2 = (0 - 0{,}8635) \cdot 0{,}1179 = -0{,}1018$; $\delta_2^2 = (1 - 0{,}1107) \cdot 0{,}0985 = 0{,}0876$
5. porównaj z rozdz. 5: tam $\delta^2 = \mp 0{,}98$, tutaj ok. $0{,}1$. Czynnik f' zmniejszył błąd prawie 10 razy.
6. nowa waga: $w_{11}^2 = 2 + 0{,}5 \cdot (-0{,}1018) \cdot 0{,}9820 = 2 - 0{,}0500 = 1{,}9500$ (przy CE było $1{,}5185$)
7. po jednym kroku (wszystkie wagi): E spada z $0{,}7682$ do $0{,}7498$, czyli tylko o 2%. Przy CE strata spadła o 47%.

Dlaczego tak (ważne na wyjściówkę): gdy wyjście z sigmoidą jest pewne i się myli (np. $\hat{y} = 0{,}98$, a powinno być 0), to $f'(z) = 0{,}98 \cdot 0{,}02 = 0{,}0196$. MSE daje wtedy $\delta = -0{,}98 \cdot 0{,}0196 = -0{,}019$, a entropia krzyżowa $\delta = -0{,}98$. Czyli MSE uczy się **najwolniej wtedy, gdy myli się najbardziej**. Dlatego do klasyfikacji używa się entropii krzyżowej.

Sprawdź się 6: jedno wyjście z sigmoidą, $\hat{y} = 0{,}5$, prawda $y = 1$. Policz δ dla MSE i dla entropii krzyżowej. Przy której stracie i ile razy delta jest większa?

7. Zapis macierzowy (wykład 4, slajdy 17–19)

7.1. Oznaczenia (slajd W4: 17)

symbol	co to jest	wymiar
$\mathbf{x}$	wzorzec wejściowy, wektor kolumna	$N \times 1$
$\mathbf{z}^l$	pobudzenia wszystkich neuronów warstwy l	$n_l \times 1$
$\mathbf{a}^l$	aktywacje warstwy l; $\mathbf{a}^1 = \mathbf{x}$ (W4 numeruje wejście jako warstwę 1)	$n_l \times 1$
W^l	wagi między warstwą $l-1$ a l; wiersz = neuron warstwy l	$n_l \times n_{l-1}$
$\mathbf{b}^l$	biasy warstwy l	$n_l \times 1$
$\boldsymbol{\delta}^l$	błędy warstwy l, tu $\boldsymbol{\delta}^l = \frac{\partial C}{\partial \mathbf{z}^l}$ (gradient, więc znak „odpowiedź minus prawda”)	$n_l \times 1$
C	funkcja kosztu (= $\mathcal{L}$)	liczba
$\odot$	iloczyn Hadamarda (element po elemencie)	–
L	**tu**: numer ostatniej warstwy	–

7.2. Algorytm (slajd W4: 19)

1. **Wejście:** $\mathbf{a}^1 = \mathbf{x}$
2. **W przód**, dla $l = 2, 3, \ldots, L$: $\quad \mathbf{z}^l = W^l \mathbf{a}^{l-1} + \mathbf{b}^l, \quad \mathbf{a}^l = f(\mathbf{z}^l)$
3. **Błąd na wyjściu:** $\quad \boldsymbol{\delta}^L = \dfrac{\partial C}{\partial \mathbf{a}^L} \odot f'(\mathbf{z}^L)$
4. **Wstecz**, dla $l = L-1, \ldots, 2$: $\quad \boldsymbol{\delta}^l = \left((W^{l+1})^T \boldsymbol{\delta}^{l+1}\right) \odot f'(\mathbf{z}^l)$
5. **Aktualizacja**, dla $l = L, \ldots, 2$: $\quad W^l \leftarrow W^l - \alpha \sum_{\mathbf{x}} \boldsymbol{\delta}^l_{\mathbf{x}} (\mathbf{a}^{l-1}_{\mathbf{x}})^T, \quad \mathbf{b}^l \leftarrow \mathbf{b}^l - \alpha \sum_{\mathbf{x}} \boldsymbol{\delta}^l_{\mathbf{x}}$

Co oznacza każdy symbol: $\frac{\partial C}{\partial \mathbf{a}^L}$ – pochodna kosztu po wyjściu sieci; $(W^{l+1})^T$ – transponowana macierz wag następnej warstwy (wiersze zamienione z kolumnami); $\sum_{\mathbf{x}}$ – suma po wzorcach w paczce (przy SGD jeden wzorzec); $\boldsymbol{\delta}(\mathbf{a})^T$ – kolumna razy wiersz = macierz (iloczyn zewnętrzny).

Jak to się ma do wzorów z rozdz. 4 (każda linijka):

1. $\sum_k \delta_k^2 w_{kj}^2$ = element j wektora $(W^2)^T \boldsymbol{\delta}^2$. Transpozycja sprawia, że idziemy „po kolumnie j” macierzy W^2, czyli po wszystkich wagach wychodzących z neuronu j.
2. „razy $f_j'^1(z_j^1)$ dla każdego j” = $\odot f'(\mathbf{z}^1)$
3. „$\delta_k \cdot a_j$ dla każdej wagi w_{kj}” = macierz $\boldsymbol{\delta}\,\mathbf{a}^T$, bo jej element (k, j) to dokładnie $\delta_k a_j$. Wymiary: $(n_l \times 1)(1 \times n_{l-1}) = n_l \times n_{l-1}$ – tyle samo co W^l (dopisek ze slajdu W4: 19).
4. minus zamiast plus, bo tu $\boldsymbol{\delta} = \hat{y} - y$ (patrz uwaga w rozdz. 0)

7.3. Przykład z rozdz. 5 macierzowo

Dane: $\mathbf{a}^1 = [0{,}9820;\ 0{,}1192]$ (wyjście warstwy ukrytej), $\hat{\mathbf{y}} = [0{,}9807;\ 0{,}0193]$, $\mathbf{y} = [0;\ 1]$, $W^2 = \begin{bmatrix} 2 & -1 \\ -2 & -1 \end{bmatrix}$, $f'(\mathbf{z}^1) = [0{,}01766;\ 0{,}1050]$, $\alpha = 0{,}5$.

1. błąd wyjścia (softmax + CE): $\boldsymbol{\delta}^2 = \hat{\mathbf{y}} - \mathbf{y} = [0{,}9807;\ -0{,}9807]$

2. transpozycja: $(W^2)^T = \begin{bmatrix} 2 & -2 \\ -1 & -1 \end{bmatrix}$

3. $(W^2)^T \boldsymbol{\delta}^2 = \begin{bmatrix} 2 \cdot 0{,}9807 + (-2)(-0{,}9807) \\ (-1) \cdot 0{,}9807 + (-1)(-0{,}9807) \end{bmatrix} = \begin{bmatrix} 3{,}9228 \\ 0 \end{bmatrix}$

4. $\odot f'$: $\boldsymbol{\delta}^1 = [3{,}9228 \cdot 0{,}01766;\ 0 \cdot 0{,}1050] = [0{,}0693;\ 0]$ – to samo co w rozdz. 5, tylko z przeciwnym znakiem ✓

5. $\boldsymbol{\delta}^2 (\mathbf{a}^1)^T = \begin{bmatrix} 0{,}9807 \\ -0{,}9807 \end{bmatrix} [0{,}9820,\ 0{,}1192] = \begin{bmatrix} 0{,}9631 & 0{,}1169 \\ -0{,}9631 & -0{,}1169 \end{bmatrix}$

6. $W^2 - 0{,}5 \cdot (\dots) = \begin{bmatrix} 2 - 0{,}4815 & -1 - 0{,}0585 \\ -2 + 0{,}4815 & -1 + 0{,}0585 \end{bmatrix} = \begin{bmatrix} 1{,}5185 & -1{,}0585 \\ -1{,}5185 & -0{,}9415 \end{bmatrix}$ – identycznie jak w 5.4 ✓

Po co macierze (W4: 20): te same liczby, ale jedno mnożenie macierzy zamiast pętli po k, j, i. Biblioteki (numpy) i karty graficzne robią to równolegle, więc jest dużo szybciej. W kodzie z ćw. 3 było dokładnie to: `dW = delta @ x.T`, `dx = W.T @ delta`.

8. Algorytm uczenia: fazy, SGD, epoki, zatrzymanie

8.1. Dwie fazy (W3: 22–24, W4: 13–14)

faza	krok	wzór
w przód (forward)	podaj wejście	$\mathbf{x}_p = [x_{p1}, \ldots, x_{pN}]^T$
	pobudzenia warstwy ukrytej	$z^1_{pj} = \sum_i w^1_{ji} x_{pi}$
	wyjścia warstwy ukrytej	$a^1_{pj} = f^1_j(z^1_{pj})$
	pobudzenia i wyjścia warstwy wyjściowej	$z^2_{pk} = \sum_{j=0}^{L} w^2_{kj} a^1_{pj}$, $\hat{y}_{pk} = f^2_k(z^2_{pk})$
wstecz (backward)	błędy wyjść	CE + softmax: $\delta^2_{pk} = y_{pk} - \hat{y}_{pk}$; MSE: $\delta^2_{pk} = (y_{pk} - \hat{y}_{pk}) f'^2_k(z^2_{pk})$
	błędy ukrytych	$\delta^1_{pj} = f'^1_j(z^1_{pj}) \sum_k \delta^2_{pk} w^2_{kj}$
	wagi wyjściowe	$w^2_{kj}(t+1) = w^2_{kj}(t) + \mu \delta^2_{pk} a^1_{pj}$
	wagi ukryte	$w^1_{ji}(t+1) = w^1_{ji}(t) + \mu \delta^1_{pj} x_{pi}$

Po ludzku: w przód sygnał idzie od wejścia do wyjścia. Wstecz błąd idzie od wyjścia do wejścia („rzutowanie błędów od warstwy wyjściowej do wejściowej"). Najpierw liczymy wszystkie δ, dopiero potem zmieniamy wagi.

8.2. Kiedy zmieniać wagi (W3: 21, 36; W4: 15)

sposób	kiedy zmiana wag	wzór	uwagi
po wzorcu, online, **SGD**	po każdym wzorcu	$\Delta w^2_{kj} = \mu\, a^1_{pj} \delta^2_{pk}$	szybkie kroki, ale „szarpane"
mini-batch	po m wzorcach (m = rozmiar paczki)	$\Delta w^2_{kj} = \mu \sum_{p=1}^{m} a^1_{pj} \delta^2_{pk}$	kompromis, standard w praktyce
po epoce, offline, **batch GD**	po wszystkich P wzorcach	$\mathcal{L} = \sum_{p=1}^{P} \mathcal{L}_p$, $\Delta w^2_{kj} = \mu \sum_{p=1}^{P} a^1_{pj} \delta^2_{pk}$	„wymaga pamiętania dużej ilości informacji"

Co oznacza każdy symbol: epoka – jedno przejście przez cały zbiór uczący; P – liczba wzorców; p – indeks wzorca.

Na liczbach: dwa wzorce, $\mu = 0{,}1$. Dla wagi w^2_{11}: wzorzec 1 daje $a^1_1 \delta^2_1 = 0{,}3$, wzorzec 2 daje $-0{,}1$.

1. po wzorcu: najpierw $w \mathrel{+}= 0{,}1 \cdot 0{,}3 = 0{,}03$, potem (już na nowej wadze liczymy nowe δ) mniej więcej $w \mathrel{+}= 0{,}1 \cdot (-0{,}1) = -0{,}01$ – dwa kroki
2. po epoce: $w \mathrel{+}= 0{,}1 \cdot (0{,}3 + (-0{,}1)) = 0{,}02$ – jeden krok, liczony na tych samych wagach dla obu wzorców

8.3. Co trzeba ustalić przed startem i kiedy stop (W3: 25, W4: 21)

- **Przed startem:** losowe wagi początkowe; współczynnik uczenia μ; warunek zatrzymania.

- **Warunki zatrzymania:** błąd dostatecznie mały dla każdej pary uczącej; przez zadaną liczbę epok błąd się nie zmniejsza; przeszła zadana liczba epok; błąd na zbiorze walidującym zaczyna rosnąć (wczesne zatrzymanie, rozdz. 11).
- **Uwagi z W4: 21:** nie ma gwarancji zbieżności – funkcja straty nie jest wypukła i ma wiele minimów lokalnych; duże sieci potrzebują dużo danych i często tysięcy epok; minimów lokalnych unika się, ucząc kilka razy z różnymi losowymi wagami początkowymi.

9. Funkcje aktywacji i ich pochodne

nazwa	wzór	pochodna	zakres
sigmoida (logistyczna)	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z))$, maks. 0,25 w $z=0$	$(0,1)$
tangens hiperboliczny	$\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$1-\tanh^2(z)$, maks. 1 w $z=0$	$(-1,1)$
ReLU	$\max(0,z)$	0 dla $z \le 0$, 1 dla $z > 0$	$[0,\infty)$
Leaky ReLU	z dla $z>0$, αz w przeciwnym razie	1 dla $z>0$, α w przeciwnym razie	$(-\infty,\infty)$
softplus	$\ln(1+e^{z})$	$\frac{1}{1+e^{-z}} = \sigma(z)$	$(0,\infty)$

Na liczbach (Leaky ReLU z $\alpha = 0{,}1$):

z	σ	σ'	tanh	tanh'	ReLU	ReLU'	Leaky	Leaky'	softplus	softplus'
-2	0,119	0,105	$-0{,}964$	0,071	0	0	$-0{,}2$	0,1	0,127	0,119
$-0{,}5$	0,378	0,235	$-0{,}462$	0,786	0	0	$-0{,}05$	0,1	0,474	0,378
0	0,5	0,25	0	1	0	0	0	0,1	0,693	0,5
$0{,}5$	0,622	0,235	0,462	0,786	0,5	1	0,5	1	0,974	0,622
2	0,881	0,105	0,964	0,071	2	1	2	1	2,127	0,881

9.1. Sigmoida i tanh (W4: 23–24)

Po ludzku: obie mają kształt litery S i „nasycają się”: dla dużych $|z|$ są płaskie, a pochodna jest prawie 0. Wyjście sigmoidy można czytać jako prawdopodobieństwo klasy albo „zapalenie neuronu” po przyjęciu progu. Tanh jest symetryczna względem 0 (daje liczby ujemne i dodatnie), ma większą pochodną (do 1 zamiast do 0,25) i „mapuje wartości odstające w kierunku średniej”. W głębokich sieciach w warstwach ukrytych sigmoida jest rzadko używana, tanh jest lepsza.

Skąd $\tanh(x) = 2\sigma(2x) - 1$ („tanh to przeskalowana sigmoida”):

1. $2\sigma(2x) - 1 = \dfrac{2}{1+e^{-2x}} - 1$
2. wspólny mianownik: $= \dfrac{2-(1+e^{-2x})}{1+e^{-2x}} = \dfrac{1-e^{-2x}}{1+e^{-2x}}$
3. mnożę górę i dół przez e^{x} (to nie zmienia ułamka): $= \dfrac{e^{x}-e^{-x}}{e^{x}+e^{-x}} = \tanh(x)$ ✓
4. na liczbach: $\tanh(0{,}5) = 0{,}4621$; $2\sigma(1) - 1 = 2 \cdot 0{,}7311 - 1 = 0{,}4621$ ✓

Skąd $\tanh' = 1 - \tanh^2$: $u = e^{z} - e^{-z}$, $v = e^{z} + e^{-z}$. Pochodna e^{-z} to $-e^{-z}$, więc $u' = e^{z} + e^{-z} = v$ i $v' = e^{z} - e^{-z} = u$.

5. wzór na iloraz: $\tanh' = \dfrac{u'v - uv'}{v^2} = \dfrac{v \cdot v - u \cdot u}{v^2} = 1 - \dfrac{u^2}{v^2} = 1 - \tanh^2(z)$

9.2. ReLU (W4: 26–27)

Zalety: chroni przed zanikającym gradientem (pochodna dla $z > 0$ wynosi dokładnie 1, więc nie zmniejsza błędu); model uczy się znacznie szybciej (szybka aktywacja i pochodna, brak tłumienia sygnału, gdy neuron jest aktywny); jest prosta obliczeniowo (to tylko porównanie z zerem). **Wady:** może prowadzić do eksplodującego gradientu (rozdz. 10); mogą istnieć neurony, które nigdy nie wyjdą ponad próg – dla $z \leq 0$ pochodna to 0, więc ich wagi nigdy się nie zmienią („martwe neurony").

9.3. Leaky ReLU i softplus (W4: 30–31)

Leaky ReLU: dla $z \leq 0$ zamiast 0 daje αz (np. $\alpha = 0{,}01$), więc pochodna nigdy nie jest zerem i neuron nie umiera. $\alpha = 0$ to zwykłe ReLU (słabo wykorzystane neurony), $\alpha = 1$ to funkcja liniowa $y = z$ (utrata nieliniowości, a ta jest potrzebna – bez niej cała sieć to jedna funkcja liniowa).

Softplus: „wygładzone ReLU". Pochodna jest określona wszędzie (ReLU nie ma pochodnej w samym $z = 0$, ma tam „ostry róg"). Dla $z \to -\infty$ pochodna dąży do 0.

Skąd pochodna softplus to sigmoida:

1. $(\ln u)' = \frac{u'}{u}$, gdzie $u = 1 + e^z$, $u' = e^z$: $\frac{d}{dz}\ln(1+e^z) = \frac{e^z}{1+e^z}$
2. dzielę górę i dół przez e^z: $= \frac{1}{e^{-z}+1} = \sigma(z)$

Sprawdź się 9: (a) policz $\tanh(1)$ ze wzoru $2\sigma(2) - 1$ (masz $\sigma(2) = 0{,}881$) i jej pochodną. (b) Neuron z ReLU ma $z = -3$ dla wszystkich wzorców. O ile zmienią się jego wagi? A z Leaky ReLU, $\alpha = 0{,}01$?

10. Zanikający i eksplodujący gradient (W4: 25–29)

10.1. Zanikający gradient

Skąd się bierze: wzór z rozdz. 4: $\delta_j^1 = f'(z_j^1)\sum_k \delta_k^2 w_{kj}^2$. Każda warstwa w drodze wstecz **mnoży** błąd przez f'. Sigmoida ma $f' \leq 0{,}25$, a w nasyceniu (bardzo duże lub bardzo małe z, aktywacja prawie 0 lub 1) $f' \approx 0$. Po kilku warstwach błąd jest tak mały, że pierwsze warstwy prawie się nie uczą: „sygnał prawie nie płynie przez połączenia". Duże wagi początkowe pogarszają sprawę, bo dają duże $|z|$ i wpychają neurony w nasycenie.

Na liczbach (najlepszy przypadek sigmoidy, $f' = 0{,}25$ w każdej warstwie, wagi 1):

1. 1 warstwa: $0{,}25$
2. 2 warstwy: $0{,}25^2 = 0{,}0625$
3. 5 warstw: $0{,}25^5 = 0{,}00098$
4. 10 warstw: $0{,}25^{10} = 0{,}00000095$ – błąd w pierwszej warstwie jest milion razy mniejszy niż na wyjściu
5. w rozdz. 5.3 widać to w małej skali: neuron w nasyceniu ($z = 4$) miał $f' = 0{,}018$ i dostał $\delta = -0{,}07$ zamiast ok. $-3{,}9$

Rozwiązanie (W4: 26): ReLU – dla $z > 0$ pochodna wynosi 1, więc nie zmniejsza błędu. Pomaga też dobra inicjalizacja wag (He, NAUKA_cw3 rozdz. 10).

10.2. Eksplodujący gradient

Skąd się bierze (W4: 28): przy ReLU $\frac{\partial y}{\partial z} = 1$ (gdy neuron aktywny), więc przy przejściu przez neuron błąd jest mnożony tylko przez wagę: $\frac{\partial y}{\partial x} = w_{yx}$ (albo 0, gdy $y = 0$). Jeśli suma tych mnożników jest większa od 1 ($|\sum_y \frac{\partial y}{\partial x}| > 1$), błąd **rośnie** z każdą warstwą. Gradient w pierwszych warstwach jest olbrzymi, kroki są olbrzymie, wagi „wybuchają" (często do NaN).

Na liczbach (każda warstwa mnoży błąd przez 1,5):

1. 5 warstw: $1{,}5^5 = 7{,}6$
2. 10 warstw: $1{,}5^{10} = 57{,}7$
3. 20 warstw: $1{,}5^{20} = 3325$

10.3. Jak się bronić (W4: 29)

$$\text{obcięcie: } \Delta w_{xy} \sim \max\left(-\Delta_0,\ \min\left(\Delta_0,\ \frac{\partial E}{\partial w_{xy}}\right)\right) \qquad \text{znak: } \Delta w_{xy} \sim \operatorname{sign}\left(\frac{\partial E}{\partial w_{xy}}\right)$$

Co oznacza każdy symbol: Δ_0 – próg (np. 1); $\min(\Delta_0, g)$ – nie więcej niż Δ_0; $\max(-\Delta_0, \dots)$ – nie mniej niż $-\Delta_0$; $\operatorname{sign}(g)$ – znak: $+1$ dla $g > 0$, -1 dla $g < 0$; $\sim$ – „proporcjonalne do".

Na liczbach, $\Delta_0 = 1$:

1. $g = 5$: $\min(1, 5) = 1$, $\max(-1, 1) = 1$ → obcięte do 1; $\operatorname{sign}(5) = 1$
2. $g = -3$: $\min(1, -3) = -3$, $\max(-1, -3) = -1$ → obcięte do -1; $\operatorname{sign}(-3) = -1$

3. $g = 0{,}4$: $\min(1; 0{,}4) = 0{,}4$, $\max(-1; 0{,}4) = 0{,}4$ → bez zmian; $\operatorname{sign}(0{,}4) = 1$

4. $g = 0{,}0001$: obcięcie nic nie zmienia, ale $\operatorname{sign} = 1$ – dlatego znak pomaga też na **zanikający** gradient: krok ma zawsze tę samą wielkość, niezależnie od tego, jak mały jest gradient

11. Walidacja: na czym uczyć, na czym sprawdzać

Cel (W3: 26): nie chodzi o wysoką dokładność na danych uczących, tylko na **nowych, wcześniej niewidzianych** wzorcach. To się nazywa zdolność do uogólniania (generalizacja).

zbiór	do czego
uczący (trenujący)	cechy x i etykiety y służą do znalezienia wag (minimalizacja straty)
walidujący	wydzielona część danych uczących; służy do wyboru hiperparametrów i modelu: uczymy wiele modeli (np. jeden na każdą wartość μ albo liczbę neuronów) i wybieramy ten, który ma najlepszy wynik na walidującym
testowy	używany **raz, na końcu**: ocena wybranego modelu = oczekiwane działanie na nowych danych. Etykiety tylko do sprawdzenia, nie do uczenia.

Hiperparametr: ustawienie, którego sieć nie uczy się sama, tylko my je wybieramy: μ, liczba warstw i neuronów, rozmiar paczki, liczba epok.

11.1. Strategie (W3: 27–28)

pomysł	ocena	dlaczego
1. wybierz hiperparametry najlepsze na danych uczących	źle	wygra model, który zapamiętał dane (przeuczenie)
2. podziel na uczące i testowe, wybierz najlepsze na testowym	źle	test został użyty do wyboru, więc nie wiadomo, jak model zadziała na naprawdę nowych danych
3. podziel na uczące, walidujące i testowe; wybierz na walidującym, oceń na testowym	lepiej	test jest nietknięty do samego końca
4. walidacja krzyżowa: podział na K foldów (części), każdy po kolei jest walidującym, wyniki uśredniamy	dobre dla małych zbiorów	dla dużych zbiorów i głębokich modeli za drogie (K razy uczenie)

Na liczbach (walidacja krzyżowa): 100 wzorców, $K = 5$ foldów po 20.

1. runda 1: uczę na foldach 2–5 (80 wzorców), sprawdzam na foldzie 1 (20 wzorców): dokładność 0,80
2. rundy 2–5 tak samo, z innym foldem do sprawdzania: 0,85; 0,78; 0,82; 0,85
3. wynik: średnia $\frac{0{,}80+0{,}85+0{,}78+0{,}82+0{,}85}{5} = \frac{4{,}10}{5} = 0{,}82$. Każdy wzorzec był raz walidującym.

Dlaczego to w ogóle działa (W3: 29): zakładamy, że dane uczące i testowe pochodzą z tego samego rozkładu $P(X, Y)$ (pary (x_n, y_n) są losowane z tego samego „źródła"). Wtedy model dobry na próbce uczącej ma szansę być dobry na testowej.

11.2. Dokładność i krzywe uczenia (W4: 56–57)

$$\text{accuracy (dokładność)} = \frac{\text{liczba poprawnie sklasyfikowanych wzorców}}{\text{liczba wszystkich wzorców}}$$

Jak czytać wykresy: (1) dokładność na uczącym i walidującym w kolejnych epokach – **duża różnica = przeuczenie** (np. 0,99 na uczącym, 0,80 na walidującym). (2) Strata w kolejnych epokach przy różnym μ: za duże μ – strata skacze albo rośnie; trochę za duże – szybko spada i utyka wysoko; za małe – spada bardzo powoli, prawie liniowo; dobre – szybko spada i dalej powoli maleje.

12. Bias i wariancja, przeuczenie i niedouczenie (W4: 32–55)

12.1. Intuicja

pojęcie	co to znaczy
bias (obciążenie)	błąd systematyczny: model jest za prosty i średnio trafia obok prawdy (np. prosta dopasowana do paraboli). Mierzy średnią dokładność.
wariancja	niestabilność: model uczony na innym zbiorze danych daje zupełnie inną odpowiedź. Mierzy spójność.
pojemność modelu	jak skomplikowane funkcje model umie przedstawić; zwykle rośnie z liczbą parametrów (wag)
brzytwa Ockhama	z dwóch wyjaśnień lepsze jest prostsze; im bardziej złożony model, tym większe ryzyko przeuczenia
niedouczenie (underfitting)	model za prosty, nie wyłapuje użytecznych zależności; **wysoki bias**; słabo i na uczącym, i na walidującym
przeuczenie (overfitting)	model za złożony, zapamiętuje wzorce i szum zamiast trendu; **wysoka wariancja**; świetnie na uczącym, słabo na nowych danych. Np. za dużo parametrów w stosunku do liczby danych.

12.2. Rozkład błędu (W4: 39)

$$\mathrm{MSE} = \mathbb{E}_{D,\epsilon}\left[\left(y - \hat{f}_D(x)\right)^2\right] = \sigma^2 + \underbrace{\left(f(x) - \mathbb{E}_D\left[\hat{f}_D(x)\right]\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_D\left[\left(\hat{f}_D(x) - \mathbb{E}_D[\hat{f}_D(x)]\right)^2\right]}_{\text{Variance}}$$

Co oznacza każdy symbol: $f(x)$ – prawdziwa funkcja (której nie znamy); $y = f(x) + \epsilon$ – zmierzona wartość z szumem ϵ; D – zbiór uczący (losowy: mogliśmy dostać inny); $\hat{f}_D$ – model nauczony na zbiorze D; $\mathbb{E}_D[\dots]$ – średnia po wielu możliwych zbiorach uczących; $\mathbb{E}_D[\hat{f}_D(x)]$ – „średni model”; σ^2 – wariancja szumu w danych, błąd nieusuwalny.

Po ludzku, składnik po składniku: (1) σ^2 – szum w danych, żaden model go nie usunie. (2) Bias² – o ile średni model chybia prawdy. (3) Variance – jak bardzo pojedyncze modele rozrzucają się wokół średniego modelu.

Na liczbach: w punkcie x prawda to $f(x) = 5$, szum ma $\sigma^2 = 0{,}5$. Uczymy ten sam model na 3 różnych zbiorach i dostajemy odpowiedzi 2, 4, 6.

1. średni model: $\frac{2+4+6}{3} = 4$
2. Bias²: $(5-4)^2 = 1$
3. Variance: $\frac{(2-4)^2+(4-4)^2+(6-4)^2}{3} = \frac{4+0+4}{3} = 2{,}67$
4. oczekiwany MSE: $0{,}5 + 1 + 2{,}67 = 4{,}17$
5. sprawdzenie wprost: średni kwadrat odległości od prawdy $\frac{(5-2)^2+(5-4)^2+(5-6)^2}{3} = \frac{9+1+1}{3} = 3{,}67$, plus szum $0{,}5 = 4{,}17$ ✓

12.3. Kompromis (W4: 41–51)

Po ludzku: żeby zmniejszyć MSE, trzeba zmniejszyć i bias, i wariancję. Ale gdy bias maleje, wariancja rośnie, i odwrotnie. Bias zmniejszamy, zwiększając złożoność modelu; wariancję – zmniejszając ją. Na wykresie błędu od złożoności modelu: bias² maleje, wariancja rośnie, ich suma (MSE) ma minimum w „optymalnej złożoności". Kompromis wyznaczamy na zbiorze walidującym/testowym (błąd uczący zawsze maleje ze złożonością, testowy ma kształt litery U). **Więcej danych** obniża wariancję, więc optymalna złożoność przesuwa się w prawo – dlatego głębokie sieci (niski bias) potrzebują ogromnych zbiorów, czego np. w medycynie często brak.

bias	wariancja	co to znaczy
niski	niska	model idealny (rzadki; w praktyce „rozsądny" bias i „rozsądna" wariancja)
niski	wysoka	przeuczenie: średnio dobrze, ale predykcje niespójne; za dużo parametrów
wysoki	niska	niedouczenie: spójnie, ale średnio źle; za mało parametrów albo słabo wyuczony
wysoki	wysoka	niespójnie i niedokładnie

Przykład „czy koszykarz?" (W4: 47): dane: wzrost i waga, klasa tak/nie. Model z gładką granicą jest dobry. Model, którego granica owija się wokół każdej pojedynczej kropki, ma 100% na danych uczących, ale nauczył się szumu i wyjątków – przeuczenie.

Skąd w ogóle przeuczenie (W4: 55): dane uczące zawierają prawdziwe prawidłowości, ale też szum, niepewne etykiety i przypadkowe regularności z błędu próbkowania. Model nie umie odróżnić prawdziwych regularności od przypadkowych i dopasowuje się do obu.

Sprawdź się 12: (a) prawda $f(x) = 3$, trzy modele dają 3, 3, 3. Policz bias² i wariancję. (b) Prawda $f(x) = 3$, modele dają 1, 5, 3. Policz bias² i wariancję. Który model jest przeuczony, a który niedouczony, jeśli w (a) prawdziwe było $f(x) = 6$? (c) Dokładność: 0,98 na uczącym, 0,70 na walidującym. Co się dzieje i co zrobić?

13. Ściąga

co	wzór / fakt
softmax	$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$; $\frac{\partial \hat{y}_k}{\partial z_k} = \hat{y}_k(1-\hat{y}_k)$, $\frac{\partial \hat{y}_j}{\partial z_k} = -\hat{y}_j\hat{y}_k$
entropia krzyżowa	$\mathcal{L} = -\sum_k y_k \ln \hat{y}_k = -\ln \hat{y}_{\text{prawdziwej}}$
softmax + CE	$\frac{\partial \mathcal{L}}{\partial z_k} = \hat{y}_k - y_k$ (bo $\sum_k y_k = 1$)
MSE + sigmoida	$E = \frac{1}{2}\sum_k (y_k - \hat{y}_k)^2$; $\delta_k^2 = (y_k - \hat{y}_k) f'(z_k^2)$
błąd wyjścia (W3)	$\delta_{pk}^2 = y_{pk} - \hat{y}_{pk}$
błąd ukryty	$\delta_{pj}^1 = f_j'^1(z_{pj}^1) \sum_k \delta_{pk}^2 w_{kj}^2$
wagi	$w_{kj}^2 \mathrel{+}= \mu \delta_{pk}^2 a_{pj}^1$; $w_{ji}^1 \mathrel{+}= \mu \delta_{pj}^1 x_{pi}$
macierzowo	$\boldsymbol{\delta}^L = \frac{\partial C}{\partial \mathbf{a}^L} \odot f'(\mathbf{z}^L)$; $\boldsymbol{\delta}^l = ((W^{l+1})^T \boldsymbol{\delta}^{l+1}) \odot f'(\mathbf{z}^l)$; $W^l \mathrel{-}= \alpha \sum \boldsymbol{\delta}^l (\mathbf{a}^{l-1})^T$
znak delty	$y - \hat{y}$ → plus w kroku; $\hat{y} - y$ → minus w kroku. Ten sam krok.
SGD / mini-batch / batch	po 1 wzorcu / po m wzorcach / po całej epoce (suma po p)
pochodne aktywacji	$\sigma' = \sigma(1-\sigma) \le 0{,}25$; $\tanh' = 1 - \tanh^2 \le 1$; ReLU' $\in \{0,1\}$; Leaky' $\in \{\alpha, 1\}$; softplus' $= \sigma$
tanh a sigmoida	$\tanh(x) = 2\sigma(2x) - 1$
zanikający gradient	mnożenie przez $f' \le 0{,}25$ w każdej warstwie; nasycenie; lek: ReLU, dobra inicjalizacja
eksplodujący gradient	mnożenie przez wagi > 1 (np. przy ReLU); lek: obcięcie $\max(-\Delta_0, \min(\Delta_0, g))$, znak gradientu
zbiory	uczący – wagi; walidujący – wybór hiperparametrów/modelu; testowy – jedna ocena na końcu
bias–variance	MSE = σ^2 + Bias² + Variance; niedouczenie = wysoki bias, przeuczenie = wysoka wariancja
warunki stopu	błąd mały; brak poprawy przez n epok; limit epok; rośnie błąd walidacyjny

14. Pytania (jak na wyjściówce)

1. Co to jest kodowanie one-hot? Zapisz wektor y dla klasy 3 z 4.
2. Policz softmax dla $z = [2, 0]$. Która klasa jest predykcją?
3. Dlaczego dla one-hot entropia krzyżowa to $-\ln \hat{y}_{\text{prawdziwej}}$?
4. Wyprowadź $\frac{\partial \hat{y}_k}{\partial z_k}$ dla softmaxa.
5. Wyprowadź $\frac{\partial \mathcal{L}}{\partial z_k} = \hat{y}_k - y_k$. W którym miejscu korzystasz z one-hot?
6. Napisz wzór na zmianę wagi w^2_{kj} i powiedz, co znaczy każdy symbol.
7. Dlaczego we wzorze na δ^1_j jest suma po k?
8. Czym różni się δ^2 dla MSE i dla entropii krzyżowej? Która strata uczy szybciej przy pewnej, błędnej odpowiedzi i dlaczego?
9. Zapisz algorytm propagacji wstecznej macierzowo (4 linijki). Jaki wymiar ma $\boldsymbol{\delta}^l(\mathbf{a}^{l-1})^T$?
10. Dlaczego w W3 jest $+\mu\delta a$, a w zapisie macierzowym $-\alpha\delta a^T$?
11. Czym różni się SGD, mini-batch i uczenie po epoce?
12. Wymień 3 warunki zatrzymania uczenia.
13. Podaj pochodne: sigmoidy, tanh, ReLU, Leaky ReLU, softplus. Dlaczego funkcja aktywacji musi być różniczkowalna?
14. Pokaż, że $\tanh(x) = 2\sigma(2x) - 1$.
15. Co to jest zanikający gradient, skąd się bierze i jak mu zapobiec?
16. Co to jest eksplodujący gradient? Obetnij gradient $g = -7$ przy $\Delta_0 = 2$.
17. Do czego służy zbiór walidujący, a do czego testowy? Dlaczego nie wybierać hiperparametrów na testowym?
18. Na czym polega walidacja krzyżowa i kiedy się ją stosuje?
19. Zapisz rozkład MSE na bias i wariancję. Co znaczy każdy składnik?
20. Jak rozpoznać przeuczenie na wykresie dokładności? Jak rozpoznać niedouczenie?

15. Rozwiązania

Sprawdź się 1

1. warstwa 1: $z = [1 \cdot 0 + (-2) \cdot 0 + 1;\ (-1) \cdot 0 + 1 \cdot 0 + 0] = [1;\ 0]$, $a = [\sigma(1);\ \sigma(0)] = [0{,}73;\ 0{,}5]$
2. warstwa 2: $z_1 = 2 \cdot 0{,}73 - 0{,}5 = 0{,}96$, $z_2 = -2 \cdot 0{,}73 - 0{,}5 = -1{,}96$; $a = [0{,}72;\ 0{,}12]$
3. warstwa 3: $z_1 = 3 \cdot 0{,}72 - 0{,}12 - 2 = 0{,}04$, $z_2 = -0{,}72 + 4 \cdot 0{,}12 + 2 = 1{,}76$; $a = [0{,}51;\ 0{,}85]$ ✓

Sprawdź się 5

1. $\hat{y} = [0{,}9807;\ 0{,}0193]$ (wejście i wagi bez zmian), $y = [1, 0]$: $\mathcal{L} = -\ln 0{,}9807 = 0{,}0195$
2. $\delta_1^2 = 1 - 0{,}9807 = 0{,}0193$; $\delta_2^2 = 0 - 0{,}0193 = -0{,}0193$
3. $\sum_k \delta_k^2 w_{k1}^2 = 0{,}0193 \cdot 2 + (-0{,}0193)(-2) = 0{,}0772$; $\delta_1^1 = 0{,}01766 \cdot 0{,}0772 = 0{,}0014$
4. $w_{11}^2 = 2 + 0{,}5 \cdot 0{,}0193 \cdot 0{,}9820 = 2 + 0{,}0095 = 2{,}0095$
5. zmiany są małe, bo sieć już odpowiada dobrze: błąd $y - \hat{y}$ jest prawie zerem. Uczenie samo zwalnia, gdy sieć ma rację.

Sprawdź się 6

1. $f'(z) = 0{,}5 \cdot 0{,}5 = 0{,}25$
2. MSE: $\delta = (1 - 0{,}5) \cdot 0{,}25 = 0{,}125$; CE: $\delta = 1 - 0{,}5 = 0{,}5$
3. przy CE 4 razy większa (bo $f' = 0{,}25$ to i tak najlepszy możliwy przypadek sigmoidy; w nasyceniu różnica jest dużo większa)

Sprawdź się 9

(a) $\tanh(1) = 2 \cdot 0{,}881 - 1 = 0{,}762$; pochodna $1 - 0{,}762^2 = 1 - 0{,}581 = 0{,}419$

(b) ReLU: $f'(-3) = 0$, więc $\delta = 0$ i wagi zmieniają się o 0 – neuron jest „martwy”. Leaky ReLU: $f' = 0{,}01$, δ jest małe, ale niezerowe, więc wagi powoli się zmieniają i neuron może „ożyć”.

Sprawdź się 12

(a) średnia 3; Bias² $= (3 - 3)^2 = 0$; wariancja $= 0$. Gdyby prawda była 6: Bias² $= (6 - 3)^2 = 9$, wariancja 0 – **niedouczenie** (spójnie, ale źle).

(b) średnia $\frac{1+5+3}{3} = 3$; Bias² $= 0$; wariancja $= \frac{(1-3)^2 + (5-3)^2 + 0}{3} = \frac{8}{3} = 2{,}67$ – **przeuczenie** (średnio dobrze, ale każdy model inny).

(c) duża różnica między uczącym a walidującym = przeuczenie. Co zrobić: prostszy model (mniej neuronów/warstw), więcej danych, wcześniejsze zatrzymanie (gdy błąd walidacyjny zaczyna rosnąć).

Pytania z rozdz. 14 (skrótowo)

1. Jedna jedynka na miejscu prawdziwej klasy, reszta zera: $[0, 0, 1, 0]^T$.
2. $e^2 = 7{,}389$, $e^0 = 1$, suma $8{,}389$; $\hat{y} = [0{,}881;\ 0{,}119]$; predykcja: klasa 1.
3. W $-\sum_k y_k \ln \hat{y}_k$ wszystkie $y_k = 0$ poza prawdziwą klasą, więc zostaje jeden składnik.
4. Rozdz. 3.4, kroki 1–3.
5. Rozdz. 3.5, kroki 1–6; one-hot w kroku 6: $\sum_j y_j = 1$.
6. $w_{kj}^2(t+1) = w_{kj}^2(t) + \mu \delta_{pk}^2 a_{pj}^1$; symbole: rozdz. 0 i 3.5.
7. Neuron ukryty j jest połączony z każdym wyjściem k, więc wpływa na stratę przez wszystkie wyjścia; reguła łańcuchowa sumuje po wszystkich drogach.

8. MSE ma dodatkowy czynnik $f'(z_k^2)$. CE uczy szybciej, bo przy pewnej, błędnej odpowiedzi sigmoida jest w nasyceniu, $f' \approx 0$, i MSE prawie nie zmienia wag (rozdz. 6.3).
9. Rozdz. 7.2; wymiar $n_l \times n_{l-1}$, tak jak W^l.
10. W W3 $\delta = y - \hat{y} = -\frac{\partial \mathcal{L}}{\partial z}$, w zapisie macierzowym $\delta = \frac{\partial C}{\partial z}$. Przeciwny znak delty, przeciwny znak w kroku, ten sam krok.
11. Rozdz. 8.2.
12. Błąd dostatecznie mały; brak poprawy przez zadaną liczbę epok; limit epok (oraz: rosnący błąd walidacyjny).
13. Rozdz. 9; we wzorze na δ występuje f', więc musi istnieć.
14. Rozdz. 9.1, kroki 1–3.
15. Rozdz. 10.1.
16. Rozdz. 10.2; $\min(2, -7) = -7$, $\max(-2, -7) = -2$ → obcięte do -2.
17. Rozdz. 11; wybór na testowym sprawia, że test przestaje być „nowymi danymi” i jego wynik jest zbyt optymistyczny.
18. Rozdz. 11.1, pomysł 4; dla małych zbiorów.
19. Rozdz. 12.2.
20. Przeuczenie: wysoka dokładność na uczącym, dużo niższa na walidującym (rosnąca przerwa). Niedouczenie: obie niskie i blisko siebie.